ML Laboratory 01: Linear Regression

From a Fitted Line to Unseen Predictions

Table of contents

1 Objective and plan **2**

2 Before starting **2**
2.1 MATLAB survival guide . . . 2

3 Linear regression concepts **3**
3.1 The model and its parameters . . . 3
3.2 The cost function . . . 4
3.3 Training: minimize the cost . . . 5
3.4 Evaluation on unseen observations . . . 5
3.5 Summary: what is chosen, fitted, and measured? . . . 6

4 Practical work **6**
4.1 Task 1: connect the model, parameters, and cost . . . 6
4.2 Task 2: inspect cars . . . 7
4.3 Task 3: establish a baseline . . . 8
4.4 Task 4: does vehicle weight help? . . . 9
4.5 Task 5: do two more measurements help? . . . 9
4.6 Task 6: discuss and check your understanding . . . 10

5 Optional discussion — if time remains **10**

6 Function summary **10**

7 Sources **11**

1 Objective and plan

The goal of this lab is to introduce you to the fundamental concepts of linear regression. By the end of this lab, you should be able to:

- identify a dataset's predictors and target;
- distinguish a model, its adjustable parameters, and its cost function;
- explain training as minimizing cost on the training data;
- fit a linear regression model and interpret a weight;
- predict targets for observations not used to fit the model;
- compare two prescribed models against a simple baseline using test RMSE.

2 Before starting

You need **MATLAB R2019a or later**, with **Statistics and Machine Learning Toolbox** (functions `fitlm` and `predict`).

1. Keep this folder and its `data` subfolder together.
2. Open `lab1_starter.m` in MATLAB. Work in this saved file, not in an unsaved copy pasted into the Command Window.
3. Run section `0. Setup`, then work downwards. In the Editor, click a section beginning with `%%` and choose **Run Section**. Run prerequisite sections first.
4. Complete only the four expressions labelled **TODO A–D**. The `[]` and `NaN` placeholders deliberately trigger explanatory checks when reached. If you run the whole unfinished script, it stops at the first unfinished task.
5. Save your changes regularly. If you restart MATLAB or clear the workspace, rerun the earlier sections. Do not rename or move the CSV files.

2.1 MATLAB survival guide

We use a numeric matrix as a dataset, with **rows = observations**, **columns = variables**.

```
A = [10 20; 30 40; 50 60];   % Three rows, two columns
A(2,1)                       % Row 2, column 1: gives 30
A(:,2)                       % All rows of column 2: [20; 40; 60]
A(:,1:2)                     % All rows of columns 1 and 2
keep = [true; false; true];
A(keep,:)                    % Rows 1 and 3, all columns
size(A)                      % Gives [3 2]
```

- MATLAB indices start at **1**, not 0.

- A semicolon suppresses output; it does not change the calculation.
- `.^2` squares **each element** of a vector.
- All target and prediction vectors in this lab are **column vectors**.

3 Linear regression concepts

3.1 The model and its parameters

A **model** is a rule that transforms inputs into a prediction. In regression, we predict a numerical value based on one or more numerical variables (**predictors**). The predicted value aims to approximate a **target** variable.

The true target variable is denoted as y, and the model's prediction is denoted as $\hat{y}$.

In linear regression, we assume a linear relationship between the predictors and the target. For a single predictor, the model is a straight line:

$$y \approx \hat{y} = b + wx.$$

Here x is the observed input, y is the observed target, and $\hat{y}$ is the model's prediction. These are different quantities: we know y for the training examples, but want to predict it for future examples.

For three predictors, we choose

$$\hat{y} = b + w_1 x_1 + w_2 x_2 + w_3 x_3$$

and we generalize to n predictors as

$$\hat{y} = b + \sum_{i=1}^{n} w_i x_i.$$

The parameter b is the **bias** and w_i are the **weights** of the model. The bias is an additive offset; each weight controls how its predictor contributes to the prediction. Here, **vehicle weight** is an input measurement, whereas a **model weight** is a learned parameter. They are different quantities.

We use the following notations:

Quantity	Notation	Description
Input	$x_1, x_2 \dots x_n$	Input data (e.g., vehicle weight, displacement, acceleration)
Target	y	Given target value for every example
Parameters	$b, w_1, w_2, \dots, w_n$	Bias and weights, optimized during training
Prediction	$\hat{y} = b + \sum_{i=1}^{n} w_i x_i$	Calculated from the input and parameters

The three-predictor model has **four** parameters (three weights and one bias).

In general, a linear regression model with n predictors has $n + 1$ parameters.

3.2 The cost function

How do we decide whether one parameter choice is better than another? We need a single number measuring prediction errors on the training observations.

For N training observations, our **cost function** is the mean squared error:

$$\begin{aligned} J_{\text{train}}(b, w) &= \frac{1}{N} \sum_{i=1}^{N} (\hat{y}_i - y_i)^2 \\ &= \frac{1}{N} \sum_{i=1}^{N} (b + w x_i - y_i)^2 \end{aligned}$$

The data x_i, y_i stay fixed; **the cost depends on the parameters**. Lower cost means a better training fit, and zero cost means every training prediction is exact.

Real datasets contain noise, so even the best line will usually have a nonzero cost. For multiple predictors, use the same mean squared error with the corresponding model predictions.

```
ypredTrain = predict(mdl, Xtrain);
Jtrain = mean((ypredTrain - ytrain).^2);
```

3.3 Training: minimize the cost

Training means finding parameter values that minimize the training cost:

$$(b^*, w^*) = \arg\min_{b,w} J_{\text{train}}(b, w).$$

Read this as: "choose the bias and weights giving the smallest training cost." The star marks fitted values; it does not mean multiplication here. Training changes the parameters, not the chosen model family or the cost formula.

In MATLAB, `fitlm(Xtrain,ytrain)` performs ordinary least-squares training for us. It minimizes the **sum** of squared training errors. Dividing that sum by the fixed number of training rows does not change the minimizing parameters, so it also minimizes our mean squared cost. No manual parameter-search loop is needed. We are learning what optimization achieves, not implementing its algorithm today.

For N observations and n predictors, MATLAB expects `Xtrain` to be $N \times n$ and `ytrain` to be $N \times 1$.

```
mdl = fitlm(Xtrain, ytrain);     % Training: choose bias and weights
coeff = mdl.Coefficients.Estimate;
ypred = predict(mdl, Xtest);    % Prediction: use the fitted parameters
```

`fitlm` fits the bias and weights from the supplied training data. `coeff(1)` is the bias, followed by weights in predictor-column order. Prediction uses those fitted parameters; it does not train the model again. We will not interpret p-values, confidence intervals, or R^2 today.

3.4 Evaluation on unseen observations

A small training cost tells us how well we fitted known examples. It does **not** guarantee accurate predictions for new examples. Test rows are held out from fitting and used to check predictions from the already-trained model. **Do not choose the parameters by minimizing the error on the test rows.**

The car CSV contains a fixed split: **318 training rows and 80 test rows**. The split was prepared in advance, independently of the target values. Use this same split for every comparison. Neither the split flag nor the row identifier is a predictor. Do not fit a new model to the test rows.

For m test observations, we measure **root mean squared error (RMSE)**:

$$\mathrm{RMSE}_{\mathrm{test}} = \sqrt{\frac{1}{m}\sum_{i=1}^{m}(\hat{y}_i - y_i)^2}.$$

This is the square root of the test mean squared error. Unlike the training cost, we use it here only to evaluate the prescribed fitted models. It is easier to interpret because it has the same units as the target: MPG rather than MPG squared. RMSE is nonnegative; smaller is better. It is not a percentage and does not mean every individual error is below it. Do not directly compare a training MSE number with a test RMSE number: they use different scales and different observations.

```
rmseTest = sqrt(mean((ypred - ytest).^2));
```

A useful **baseline** ignores all predictors and predicts the **training mean** for every test car. Compute that mean from `ytrain`, not `ytest`. Its test RMSE provides a reference for deciding whether predictors help.

We compare only two models specified in advance. Do not repeatedly try new predictors or change the split to improve the test score: that would turn the test set into tuning data. A later lab can introduce a separate validation set.

3.5 Summary: what is chosen, fitted, and measured?

Item	What it means in this lab
Model	A straight line, or a linear combination of three predictors
Parameters	Bias and weights, adjusted during training
Training cost	Mean squared prediction error on the training rows
Training goal	Find parameters minimizing that cost, using `fitlm`
Test metric	RMSE on held-out rows, with the fitted parameters unchanged

4 Practical work

4.1 Task 1: connect the model, parameters, and cost

Goal: see how training selects one line by minimizing a numerical cost.

Run starter section 1. The bundled `synthetic_line.csv` contains 40 artificial observations generated from $y = 3 + 2x + \varepsilon$, where ε is random noise with mean 0 and standard deviation 1.5. These are not measurements of real objects; the units are arbitrary.

The supplied code compares a deliberately untrained candidate ($b = 0, w = 1$) with the trained line on the **same** 40 observations. It prints both training costs and plots both lines. There is no coding TODO in this task.

Steps

1. Inspect `size(x)` and `size(y)`. Both should be `40 1`.
2. Locate the bias and weight in the printed output. Are they near 3 and 2? Do not expect exact equality: the observations include noise.
3. Identify the model, its two parameters, and the cost being minimized. Compare the printed candidate and trained costs. Which parameter choice fits better?
4. Explain why the fitted line does not pass through every point. Would missing individual points mean that fitting failed? The generating parameters describe how noisy data were made; they need not exactly minimize the cost of this particular finite sample.

Checkpoint: one scatterplot with candidate and trained lines, two fitted parameters, and a smaller (or equal) cost for the trained line. The data-generation formula is known here; that will not be true for real data.

4.2 Task 2: inspect cars

Goal: identify predictors, targets, and the rows reserved for training/testing.

Fill in TODO A in section 2, then run the entire section 2. The bundled `auto_mpg_lab.csv` is a prepared version of UCI Auto MPG. It retains all 398 rows of the published 398-row dataset and selects three complete numerical predictors.

CSV column	Variable	Meaning / units	Use
1	`weight_1000lb`	Vehicle weight, in thousands of pounds	Predictor
2	`displacement_cuin`	Engine displacement, cubic inches	Predictor
3	`acceleration_s`	Acceleration-time measurement, seconds	Predictor
4	`mpg`	City-cycle fuel economy, miles per US gallon	Target

CSV column	Variable	Meaning / units	Use
5	`is_train`	1 = training, 0 = test	Split metadata only
6	`source_row`	One-based row number in the bundled source	Identifier only

Higher MPG means travelling farther on the same fuel quantity: **better fuel economy**, not higher fuel consumption per distance. The records describe historical cars, not today's vehicle population. No missing-value handling is required for the selected columns. Vehicle weight is divided by 1000 to make the corresponding model weight easier to interpret: it describes the predicted MPG change per 1000 lb of vehicle weight.

Steps

1. Inspect `D(1:5,:)`, `size(Xtrain)`, and `size(ytrain)`.
2. In **TODO A**, select the first predictor column of `Xtrain` as `xWeight`. Use the indexing example in the survival guide.
3. Inspect the training scatterplot. Before fitting, predict whether the model weight will be positive or negative. Is a straight line likely to be exact?

Checkpoint: `Xtrain` is `318 3`, `ytrain` is `318 1`, and `xWeight` is `318 1`. The plot shows only training observations.

4.3 Task 3: establish a baseline

Goal: obtain a reference test error before judging a trained linear model.

Run starter section 3. The baseline and its test RMSE are supplied.

Steps

1. Read the three lines computing `baselineValue`, `baselinePred`, and `baselineRMSE`.
2. State what prediction is made for every test car and why the mean uses only the training rows. Identify the baseline test RMSE in the output.

Checkpoint: one baseline test RMSE, measured in MPG. Keep this value for the comparison table; a model's error needs a reference, not just a small-looking number.

4.4 Task 4: does vehicle weight help?

Goal: train a model using vehicle weight as its only predictor, then evaluate it.

Complete TODO B and TODO C, then run starter section 4.

Steps

1. In **TODO B**, select the vehicle-weight column of `Xtest`. It must be `80 1`.
2. The fitting and prediction calls are supplied. Read them and identify which rows provide the fitted parameters and which rows receive predictions. State which cost `fitlm` minimizes: training MSE or test RMSE.
3. In **TODO C**, compute the test RMSE from `predSingle` and `ytest`, using the baseline expression as a pattern. Check that the result is a single number.
4. Compare the printed test RMSE with the baseline.
5. Read the fitted weight. Interpret its sign and units: a one-unit change in this predictor means **1000 lb**, not 1 lb. Is the bias, which predicts MPG at zero vehicle weight, physically meaningful for a real car?

Checkpoint: the training scatterplot now includes a fitted line; the model using vehicle weight has a test RMSE. Do not fit the line again using test targets.

4.5 Task 5: do two more measurements help?

Goal: compare model families while keeping the cost definition and data split fixed.

Complete TODO D, then run starter section 5.

Steps

1. In **TODO D**, use the same fitting pattern as Task 4, but provide all three columns of `Xtrain` and the same `ytrain` vector.
2. The code prints **training MSE costs** and **test RMSE values**. Identify the four parameters of the three-predictor model; what quantity was minimized?
3. Compare the three **test RMSE** values printed in the Command Window: the training-mean baseline, the vehicle-weight-only model, and the three-predictor model. Do not assume that adding predictors must improve the test result; an unchanged or worse result is valid.

Training MSE cost for the larger least-squares model should not increase, apart from numerical rounding: it can reproduce the smaller model by setting the two extra weights to zero. Test RMSE has no such guarantee. One split is a small experiment, not proof that a model will always win.

Checkpoint: identify which model has the lowest test RMSE, with all three results evaluated on the same 80 test cars. Do not rank the three predictors by raw coefficient magnitude: their

units differ and the measurements can be correlated. A fitted association is not a causal effect. These are observational records, not an experiment changing only vehicle weight: heavier cars may also differ in engine design and other characteristics. The fitted weight does not isolate what would happen if we changed one car's vehicle weight alone.

4.6 Task 6: discuss and check your understanding

Goal: explain what training optimized and what the test comparison establishes.

Use your plots and the printed results to discuss the questions below with a classmate or the instructor. These are in-lab checks: **no written lab report, figure export, or file submission is required**.

1. Interpret the model weight w when vehicle weight is the only predictor, including its sign and units. Why is it not evidence that experimentally changing vehicle weight would cause exactly this change in MPG?
2. Which prescribed model has the lowest test RMSE? Does the model using only vehicle weight beat the baseline, and do the extra predictors improve on that model in this split? Use your numbers.
3. What parameters and cost function are involved in training the model using only vehicle weight? Why can minimizing that cost still leave imperfect predictions on the test rows? Mention at least one limitation of this experiment.

This is the end of the required lab. If you are behind schedule, ask for help on the four TODO expressions; do not spend the buffer exploring additional models.

5 Optional discussion — if time remains

No additional programming or assessment is required. Could a model trained on these historical cars be trusted for a modern electric vehicle? Explain why a small error on this test set cannot establish that claim.

6 Function summary

MATLAB expression	Purpose
`readmatrix(path)`	Read a numeric CSV, detecting its header
`A(:,j)` / `A(mask,:)`	Select a column / select rows using a logical mask

MATLAB expression	Purpose
`size(A)` / `numel(y)`	Inspect matrix dimensions / number of vector elements
`fitlm(X,y)`	Choose bias and weights minimizing squared training error
`mean((ypredTrain-ytrain).^2)`	Compute training MSE cost for fixed parameters
`predict(mdl,Xnew)`	Predict for rows with the same predictor-column order
`mdl.Coefficients.Estimate`	Read coefficients: bias first
`sqrt(mean((ypred-y).^2))`	Compute RMSE on matching column vectors
`scatter(x,y)` / `plot(x,y)`	Plot observations / an ordered line
`linspace(a,b,n)'`	Make an ordered column of plotting inputs
`fullfile(...)`	Construct a platform-independent file path

7 Sources

- Quinlan, R. (1993). **Auto MPG**. UCI Machine Learning Repository. Dataset and licence, CC BY 4.0. The teaching CSV changes column selection, vehicle-weight units, and adds split/row metadata; see `data/README.md` for the exact preparation procedure.
- MathWorks: `fitlm`.
- The synthetic observations were generated specifically for this lab; the formula, seed, and reproducible offline preparation script are bundled.